IILM UNIVERSITY
GREATER NOIDA, UTTAR PRADESH

B.TECH INTERNSHIP PROJECT REPORT

Employee Management System

A Java Full-Stack Enterprise Web Application

PROJECT TYPE

Self-Directed Internship Project

STUDENT CREDENTIALS

Amit Yadav

Enrollment / Roll No.
2410030202

Program & Semester
B.Tech CSE, 5th Sem

HOST ORGANIZATION
IILM University
Self-Directed Internship Project

Agenda

What this presentation covers

1. Introduction & Problem Statement
2. Objectives of the Project
3. Technology Stack
4. System Architecture
5. Core Modules & Features
6. Role-Based Access & Security
7. REST API Design
8. Challenges & Learnings
9. Future Enhancements
10. Conclusion

Introduction & Problem Statement

Why an Employee Management System?

- Manual, paper-based or spreadsheet-driven HR processes are slow, error-prone, and hard to scale as an organization grows.
- Organizations need a single, centralized system to manage employee records, leave requests, payroll, and workforce analytics.
- The Employee Management System (EMS) automates these core HR operations through a full-stack Java web application.
- It follows an enterprise layered architecture (Controller → Service → Repository → Model) and requires zero external installation.

6
CORE MODULES

3
USER ROLES

Zero
EXTERNAL SETUP

Objectives of the Project

Centralize Employee Data

Maintain a single source of truth for employee records with full CRUD capability.

Automate Leave Workflow

Enable digital leave submission, tracking, and approval/rejection with status updates.

Streamline Payroll

Automatically calculate salary, bonuses, and deductions, and generate payslips.

Enable Data-Driven Insights

Provide real-time analytics on headcount, department distribution, and payroll spend.

Ensure Secure Access

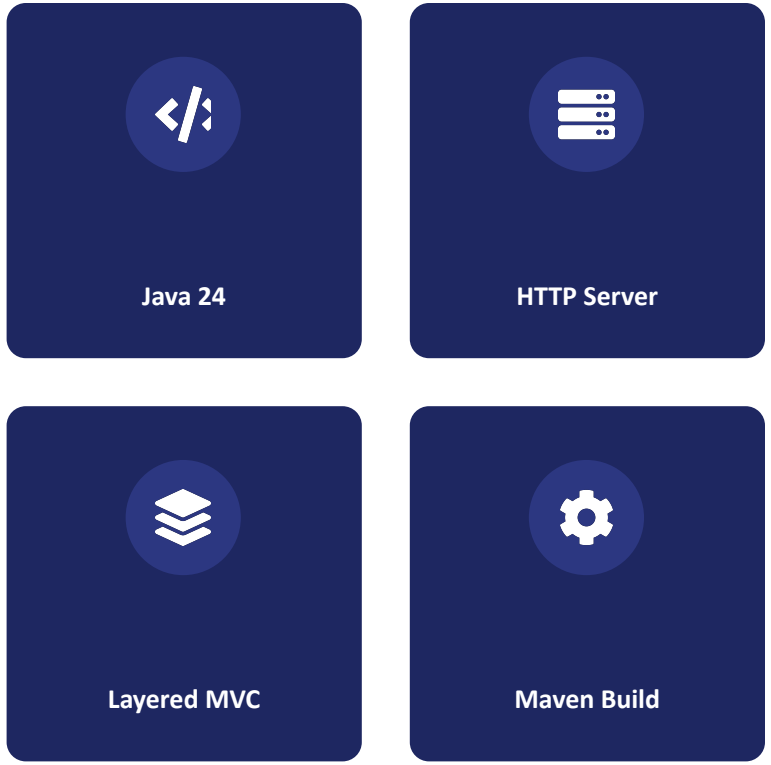
Implement role-based login for Admin, HR Manager, and Employee user types.

Support Data Portability

Allow exporting the full employee dataset to CSV for reporting and audits.

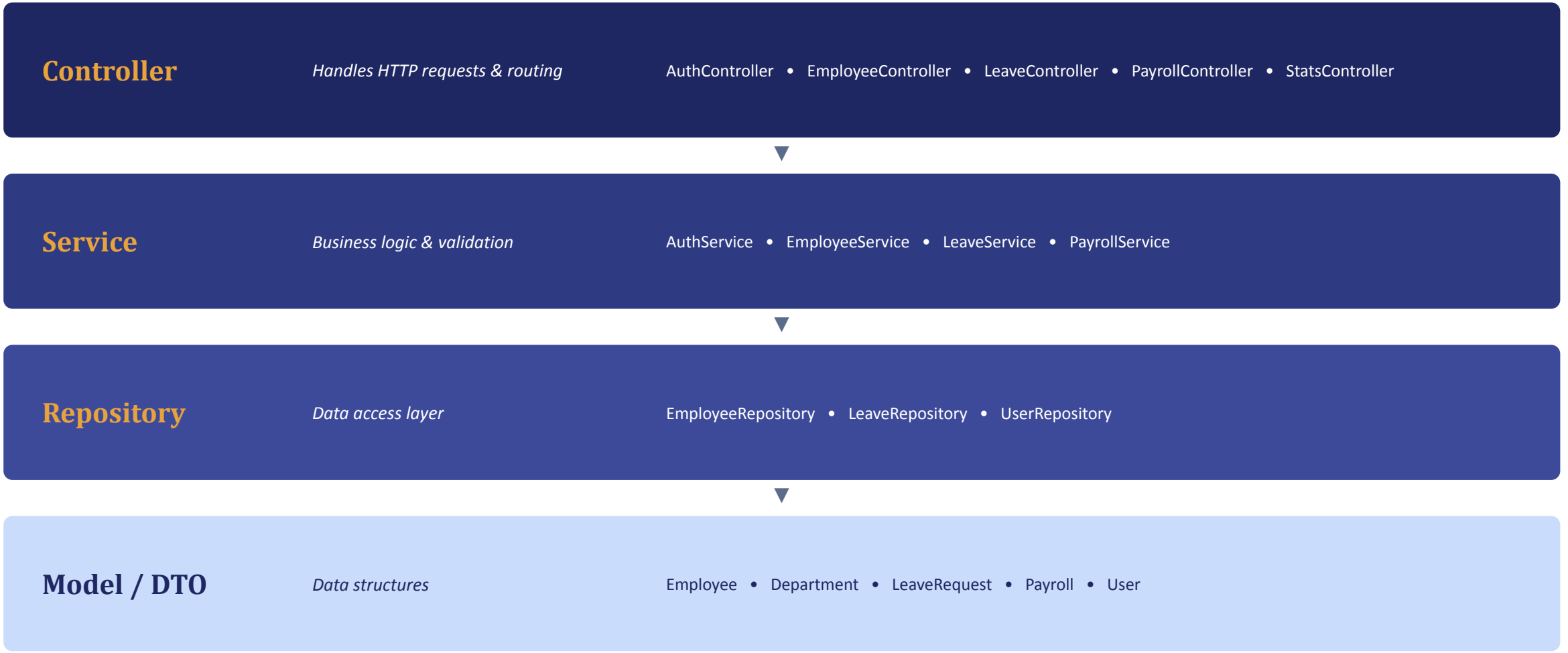
Technology Stack

Backend Language	Java 24 (Enterprise Standard Edition)
Server Engine	com.sun.net.httpserver.HttpServer (Embedded REST Server)
Architecture Pattern	Layered Enterprise: Controller → Service → Repository → Model → DTO
Data Persistence	Data file serialization with thread-safe concurrency maps
Frontend	HTML5, Vanilla CSS3 (Glassmorphism), JavaScript (ES6+)
Styling & Fonts	Plus Jakarta Sans, Outfit, Font Awesome 6
Build Tool	Maven (pom.xml)



System Architecture

Layered enterprise design pattern



Core Modules & Features

Employee Directory & CRUD

Live search, filter by department/status, add/edit/delete records.

Leave Management

Submit, track, and approve/reject leave applications by type.

Payroll & Payslips

Calculates salary, bonuses, tax deductions; generates printable payslips.

Analytics Dashboard

Real-time headcount, department distribution & payroll metrics.

Digital ID Badges

Printable ID passes with avatar, role title, and employee ID.

CSV Export

Exports the full employee dataset to a standard CSV file.

Employee Management Module

Employee Directory

DEPARTMENTS TRACKED

- Engineering
- HR
- Sales
- Finance
- Operations
- Design
- Marketing

- Full CRUD: create, view, update, and delete employee records with confirmation.
- Live real-time search by Name, Email, Position, ID, or Department.
- Filter by department: Engineering, HR, Sales, Finance, Operations, Design, Marketing.
- Filter by employment status: Active, On Leave, Terminated.
- Each employee record captures ID, contact details, department, position, salary, status, performance rating, and join date.

Leave Management Workflow

Supported Leave Types

- Annual Leave
- Sick Leave
- Casual Leave
- Maternity Leave

1

Employee Submits

Annual, Sick, Casual, or Maternity leave request submitted through the employee portal.

2

Recorded as Pending

Leave request is stored in the system with status set to PENDING.

3

Manager Reviews

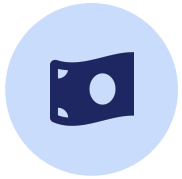
HR or Admin opens the dashboard and views all pending leave requests.

4

Approve / Reject

Status is updated; the employee's record and dashboard reflect the outcome.

Payroll & Payslip Generation



- Calculates base salary, performance-based bonuses, and applicable tax deductions.
- Computes final net pay automatically for each employee, each pay cycle.
- Disburses monthly salaries and maintains a payroll history.
- Generates a printable, previewable payslip per employee.

NET PAY FORMULA

Base Salary

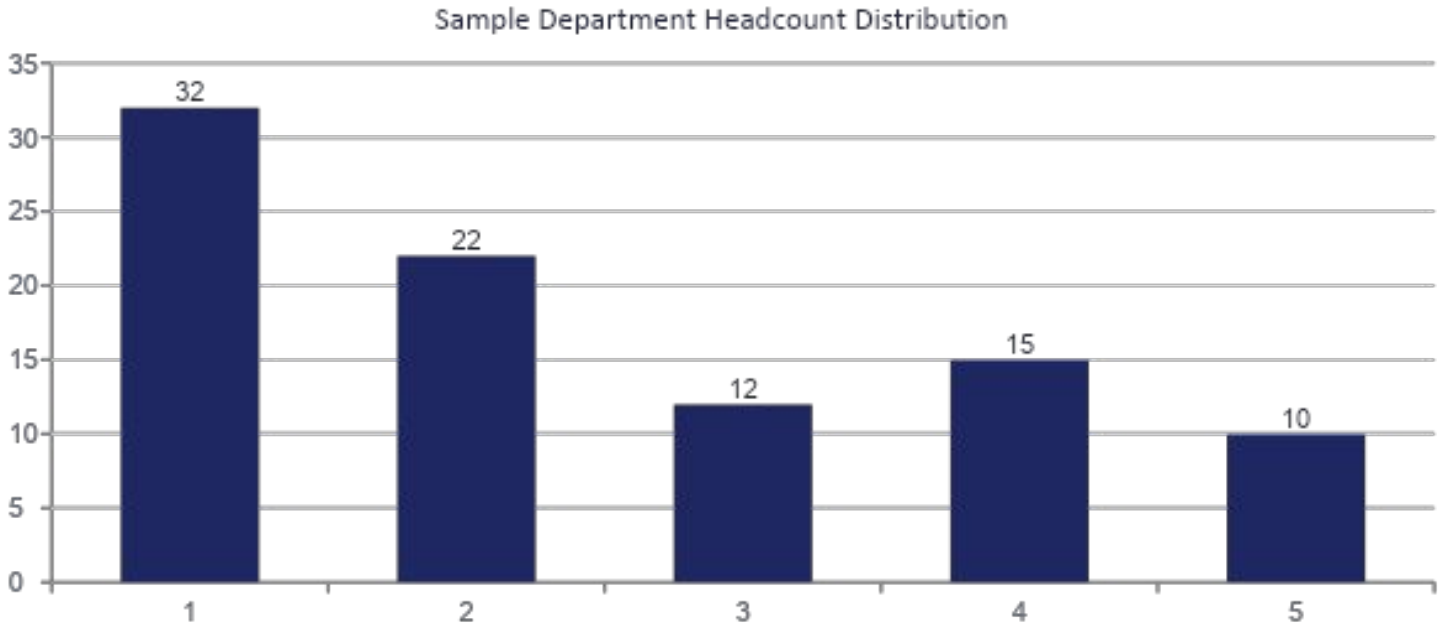
+ Performance Bonus

− Tax Deductions

= Net Pay

Analytics & Department Insights

- Real-time workforce metrics: total staff, active headcount, staff on leave.
- Total monthly payroll budget calculated dynamically from live employee data.
- Performance rating metrics summarizing employee ratings (1–5 scale).
- Visual progress indicators for department-wise headcount distribution.



Role-Based Access & Security

Three distinct roles govern access to system functionality, ensuring users only see and act on what's relevant to them.

Administrator

`admin`

Full system access — manage employees, payroll, leave, and analytics.

HR Manager

`hrmanager`

Manage employee records, review and process leave requests.

Employee

`employee`

View personal profile, submit leave requests, view payslips.

Note: default credentials shown are for demo/development use only and should be changed before any production deployment.

REST API Design

Embedded HttpServer exposes the following endpoints

METHOD	ENDPOINT	DESCRIPTION
GET	/api/employees	List employees (supports q, dept, status filters)
POST	/api/employees	Create a new employee
PUT	/api/employees/{id}	Update an existing employee
DELETE	/api/employees/{id}	Delete an employee record
GET	/api/leaves	Get all leave requests
POST	/api/leaves	Submit a new leave request
PUT	/api/leaves/{id}/status	Approve or reject a leave request
GET	/api/payroll	Get payroll history
POST	/api/payroll	Generate payslip for an employee
GET	/api/stats	Get dashboard overview analytics

Challenges Faced & Solutions

Zero-Dependency Server

Challenge: avoid external frameworks like Spring.

Solution: built the REST layer directly on com.sun.net.httpserver.HttpServer.

Concurrent Data Access

Challenge: multiple simultaneous requests reading/writing employee data.

Solution: thread-safe concurrency maps in the repository layer.

Data Persistence Without a DB

Challenge: no external database installation allowed.

Solution: custom data-file serialization for employees and leave records.

Consistent Layered Design

Challenge: keep business logic out of controllers.

Solution: strict Controller → Service → Repository separation.

Future Enhancements

- Migrate persistence layer to a relational database (PostgreSQL / MySQL) for stronger data integrity.
- Add JWT-based authentication and encrypted password storage in place of default credentials.
- Introduce email/SMS notifications for leave approvals and payslip generation.
- Build a mobile-responsive companion app for employees to apply for leave on the go.
- Add audit logging and role-based permission granularity beyond the current three roles.

Conclusion

The Employee Management System successfully demonstrates a complete, enterprise-style full-stack Java application — built without external frameworks — covering employee records, leave workflows, payroll processing, analytics, and secure role-based access. The project reinforced core software engineering principles: layered architecture, REST API design, concurrent-safe data handling, and building a polished, responsive front-end experience.

Layered Architecture **REST API** **Role-Based Security** **Real-Time Analytics**

Thank You

Questions & Discussion

Amit Yadav | Enrollment No. 2410030202 | B.Tech CSE, 5th Semester | IILM University